【STM32+LAN9252+HAL库】EtherCAT从站搭建 保姆级教程

原创 已于 2025-04-10 18:17:12 修改 · 8.8k 阅读 · 28 · 146 · CC 4.0 BY-SA版权

文章标签： #stm32 #单片机 #机器人领域 #EtherCAT #协议栈移植 #教程


EtherCAT学习 专栏收录该内容 1 篇文章 订阅专栏

该文章已生成可运行项目，预览并下载项目源码

目录

一、生成协议栈及XML文件
二、使用stm32CuboMX配置外设
三、协议栈移植

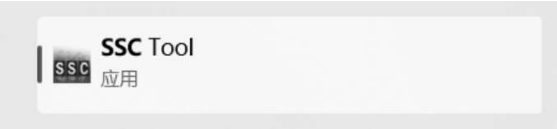
鉴于本人对EtherCAT 的掌握程度十分有限，这篇文章仅作为我搭建基础从站的过程记录不做更多讲解。本文内容主要为SPI模式的基础搭建，更多深入的学习资料和细节，大家可以通过进一步的网上搜索来获取。

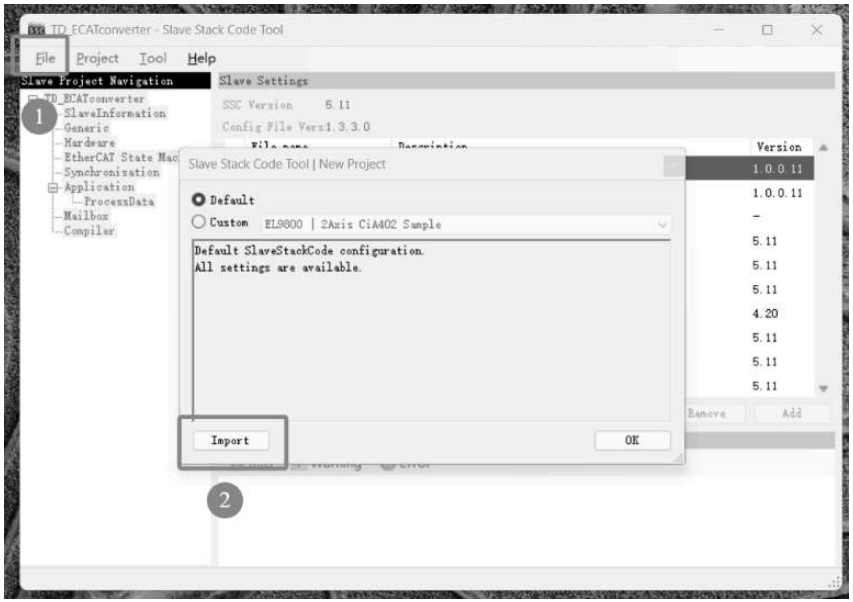
> **软件工具：**XMLNotepad2007，SSC_V5.11，TwinCAT3，Keil5，STM32CubeMX
> **资源：**lan9252-pic32-sdk_v1.1

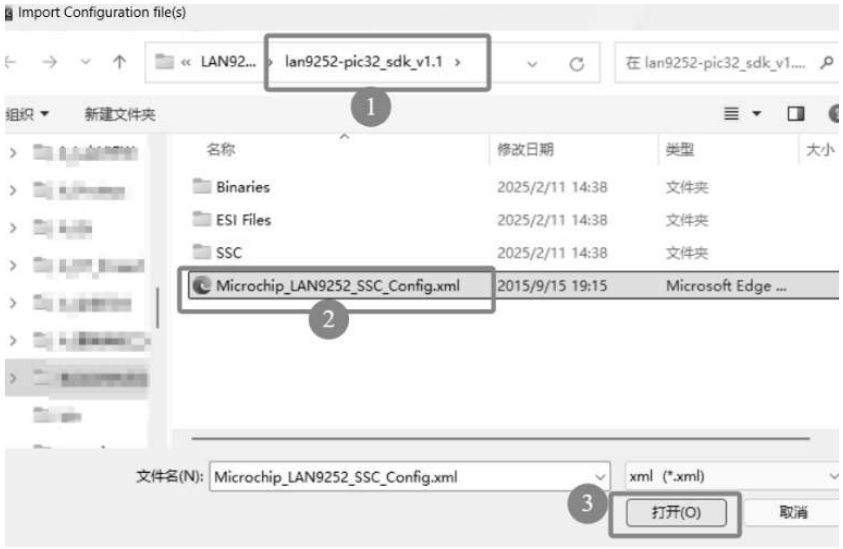
一、生成协议 栈及XML文件

我是使用的 **SSC_V5.11** 来自动生成协议栈与XML文件,期间使用到了LAN9252的官方SDK 文件 **lan9252-pic32-sdk_v1.1** 版本,网站有许多资源可以自行下载。流程可参考官方的 AN1916 文件

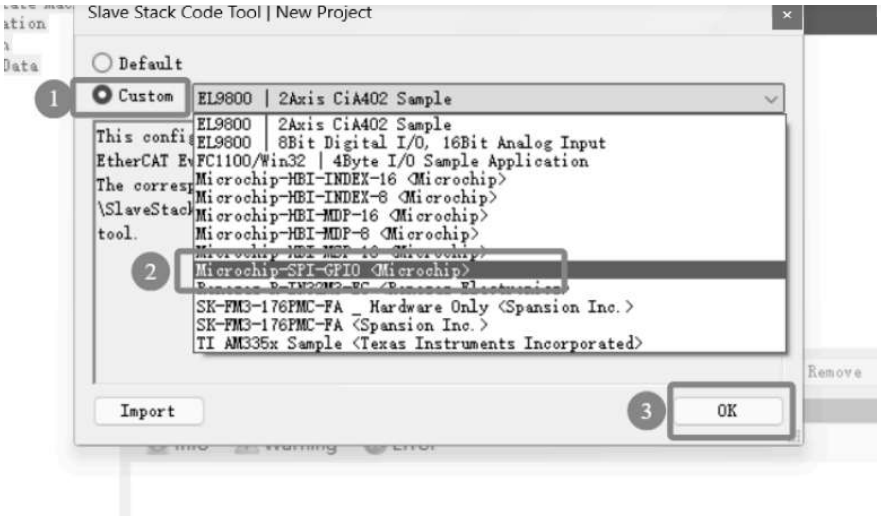
1. 下载完SSC_V5.11后找到并打开SSC Tool程序。



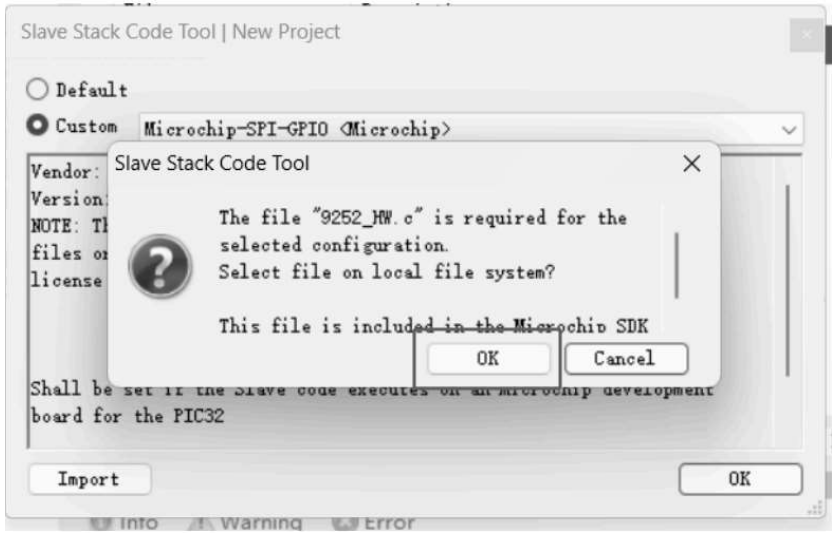


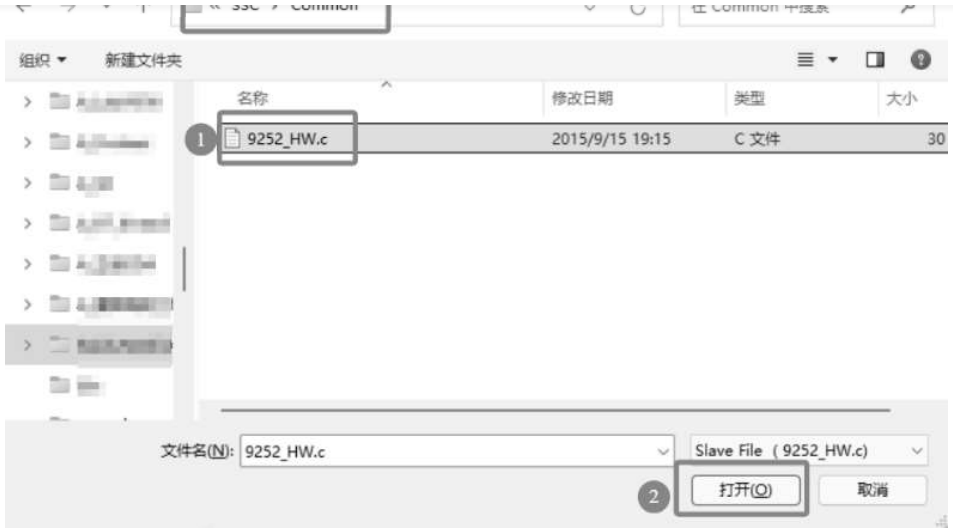


3. 选择 **Custom -> Microchip-SPI-GPIO -> OK** 使用官方模板后，XML中的一些参数就不需要自己来修改了，非常的方便。



4. 点击OK后会提示让你选取9252_HW.c文件，这个文件可以在**lan9252-pic-32_sdk_v1.1/SSC/Common文件夹下找到。**



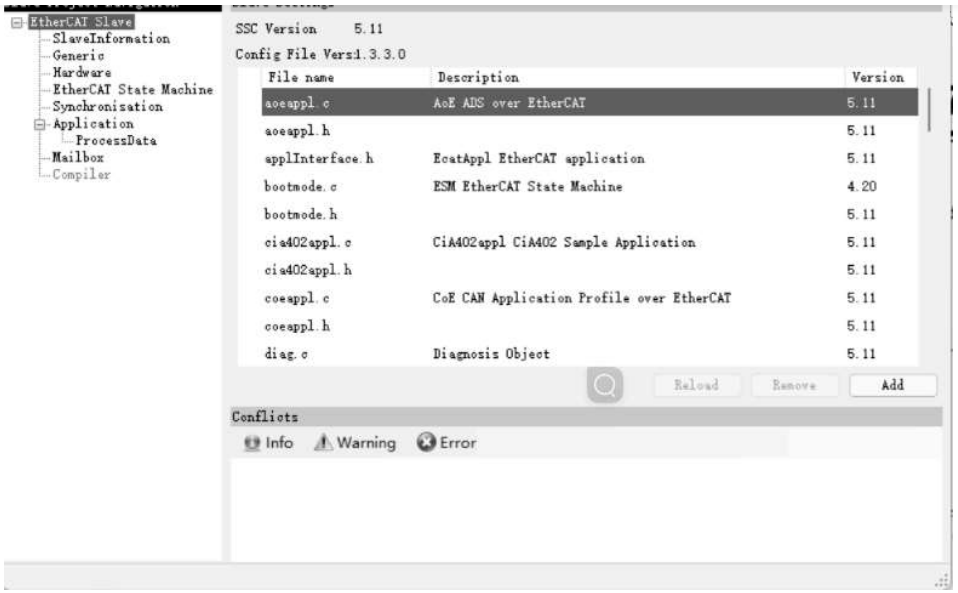


完成之后是这样的一个界面，可以按需自行更改项目信息，这里只是自己为了搭建从站进行测试，我这里先不做任何的修改。想了解里面每一项功能的可以阅读下面这篇文章

EtherCAT Slave Stack Code Tool（SSC）使用笔记-CSDN博客

文章浏览阅读1w次，点赞87次，收藏190次。学习EtherCAT有几个月了,准备做一个EtherCAT从站，摸爬滚打一路走来，遇到不少问题，这里仅作学习记录使用。移植过的平台STM32F103，STM32F407，ari32F103。从站芯片LAN9252。EtherCAT Slave Stack Code Tool (SSC) V5.12(该软件自动生成EtherCAT从站核心代码。_slave stack code tool

https://blog.csdn.net/efei123/article/details/136231104?fromshare=blogdetail&sharetype=blogdetail&sharerId=136231104&sharerefer=PC&sharesource=a1547998353&sharefrom…



5.使用Excel配置对象字典

通过选择上方工具栏 **Tool -> Application -> Create new** 来创建Excel设置通信数据，第一次创建会先提示你保存项目，自己新建一个文件夹　保存进去即可。刚刚创建的Excel如下，0x6000~0x6FFF 定义输入，0x7000~0x7FFF定义输出（这里的输入输出是以主站的角度来看的），我们可以在对应位置添加自己的对象。

				0x1011, 0x1018, 0x10F0, 0x10F1, 0x10F3, 0x1c00, 0x1c32, 0x1c33										
				- Entries less or equal one 8Bit shall not overlap byte borders.										
				- Entries greater 8Bit shall always start at an exact word border										
The object dictionary defined here shall be used complementary with ETG.5001 and ETG.1000														
Index	ObjectCode	SI	DataType	Name	Default	Min	Max	M/O/C	B/S	Access	rx/t	CoeRead	CoeWrite	Description
//0x04xx	Units													
//0x8xx	Enums (0x0800 - 0xFFF)													
//0x6nnx	Input Data of the Module (0x6000 - 0x6FFF)													
//0x7nnx	Output Data of the Module (0x7000 - 0x7FFF)													
//0x8nnx	Configuration Data of the Module (0x8000 - 0x8FFF)													
//0x9nnx	Information Data of the Module (0x9000 - 0x9FFF)													
//0xAnnx	Diagnosis Data of the Module (0xA000 - 0xAFFF)													

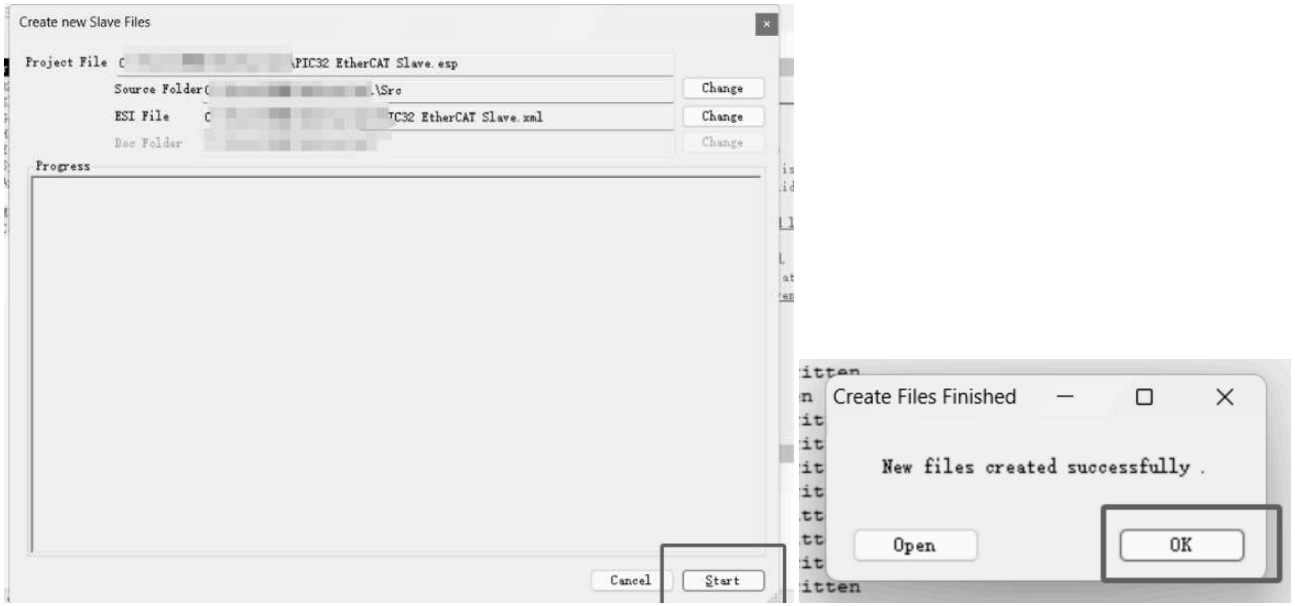
按自己的需求简单创建对象，d1用来测通讯，d2用来补位。

> 需要注意这里**一定要四字节对齐！！！一定要四字节对齐！！！一定要四字节对齐！！！**
> 因为我们后面移植协议栈时需要删除 __packed 关键字，删除之后会变成默认的四字节对齐，创建字典时如果没有字节对齐会导致与从站通讯不上，编译代码时还不会报错，到时候找问题你就找去吧，一找一个不吱声别问我咋发现的T^T。

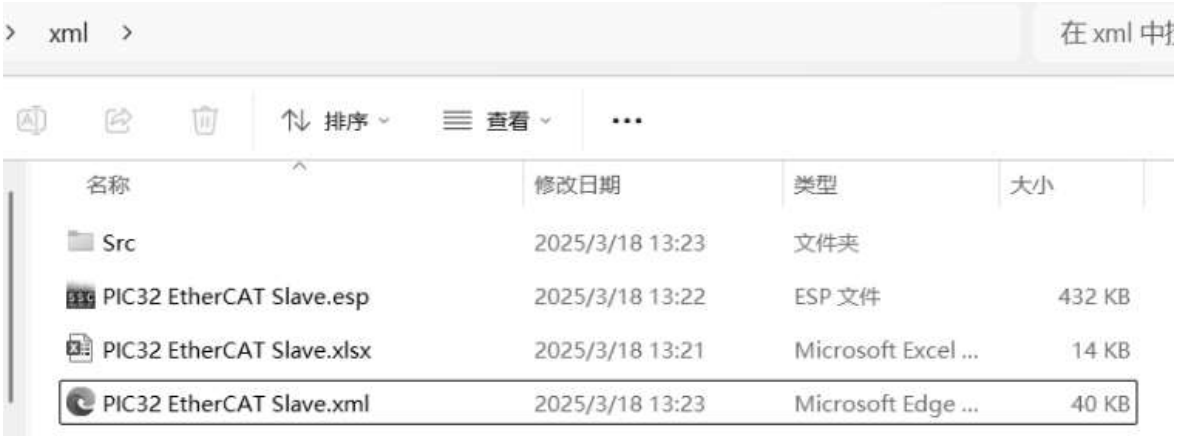
Index	ObjectCode	SI	DataType	Name	Default	Min	Max	M/O/C	B/S	Access	rx/tx	CoeRead
//0x6nnx	Input Data of the Module (0x6000 - 0x6FFF)											
0x6000	RECORD			Values								
		1	UINT16	v1						rw	tx	
		2	UINT16	v2						rw	tx	
//0x7nnx	Output Data of the Module (0x7000 - 0x7FFF)											
0x7000	RECORD			Led								
		1	UINT16	d1						rw	rx	
		2	UINT16	d2						rw	rx	
//0x8nnx	Configuration Data of the Module (0x8000 - 0x8FFF)											

添加完后保存并退出，弹出的窗口点击OK。

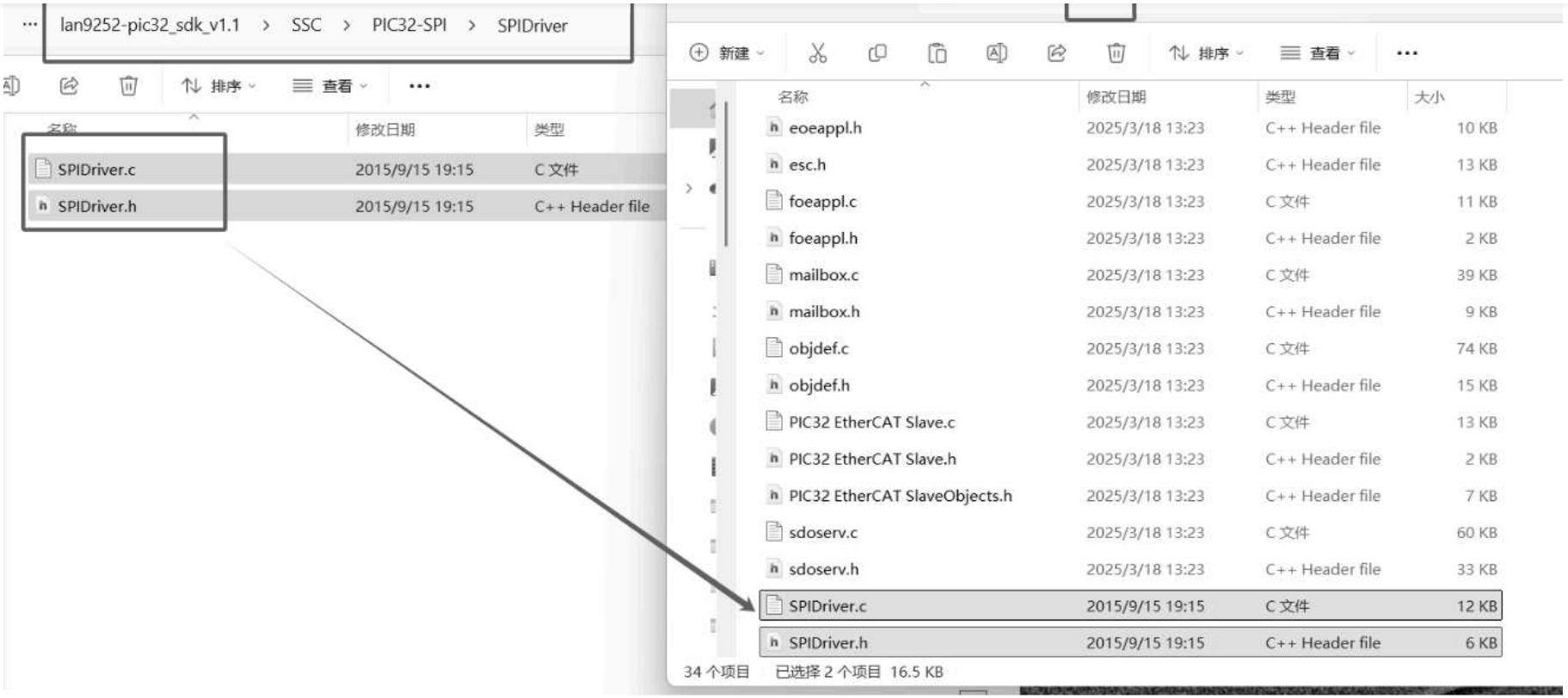
点击左上角 **Project -> Create new Slave Files -> 右下角Start -> OK ->Close** 完成之后关闭SSC Tool工具就可以了。



这时候打开刚刚工程的保存位置，看见如下四个文件说明成功了。其中 **Src** 里面保存的就是自动生成的协议栈代码，不过是基于PIC32的，后面我们将把它慢慢修改成STM32　的代码。 **.xml** 文件后续我们需要通过 TwinCAT 烧写进从站的EEPROM，想要阅读可以使用 XMLNotepad 工具，网上有很多资源可以自己按需下载。



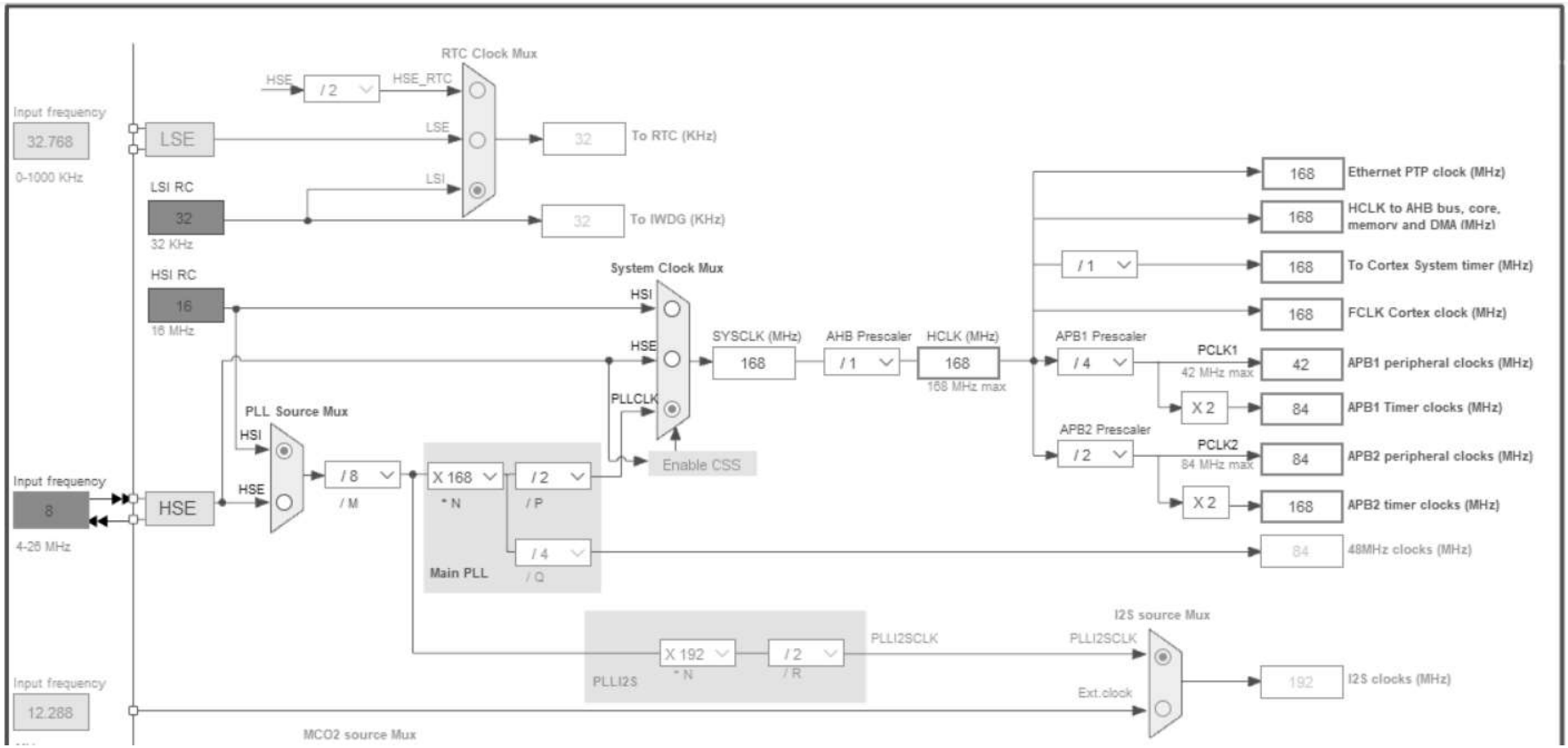
7. 官方给的SDK文件里有我们需要的SPI　驱动，将驱动的.c .h文件复制到刚刚生成的Src文件夹内。



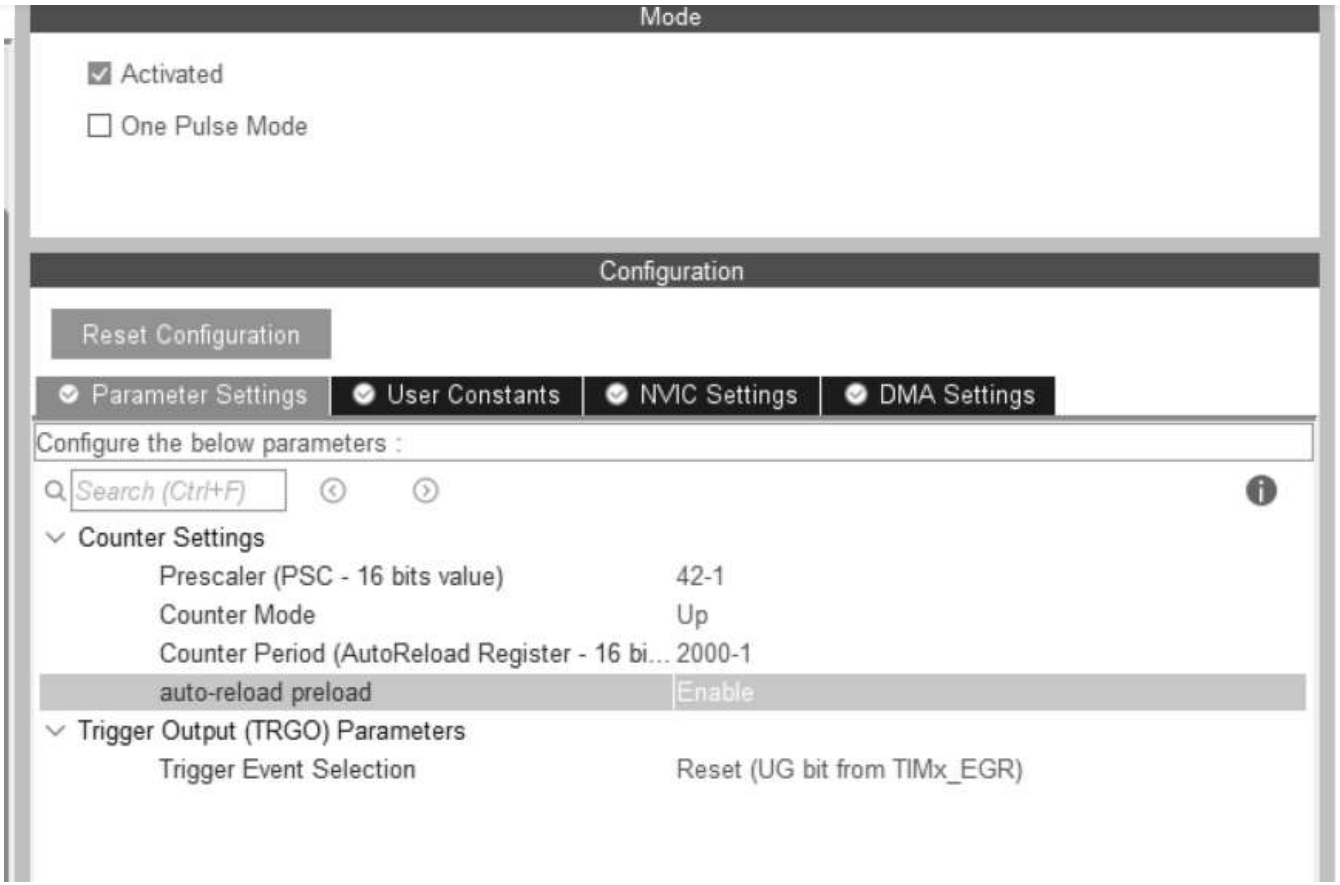
二、使用stm32CuboMX配置外设

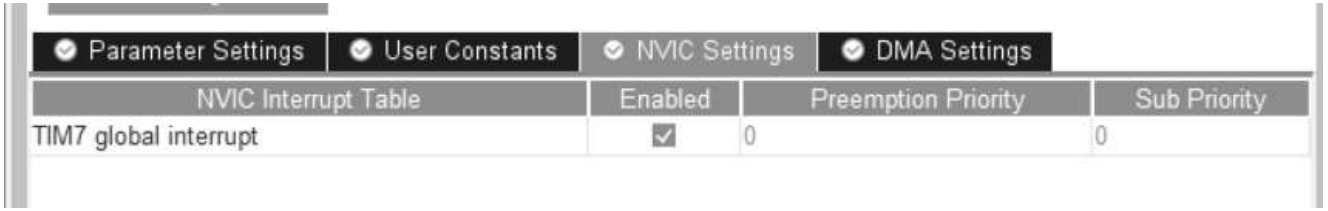
> 与LAN9252的SPI通讯需要以下几个外设：
> 1. 一组4线标准SPI
> 2. 1ms定时器
> 3. 三个外部中断： 主中断 IRQ，两个时钟同步中断 SYNC0、SYNC1

时钟树：

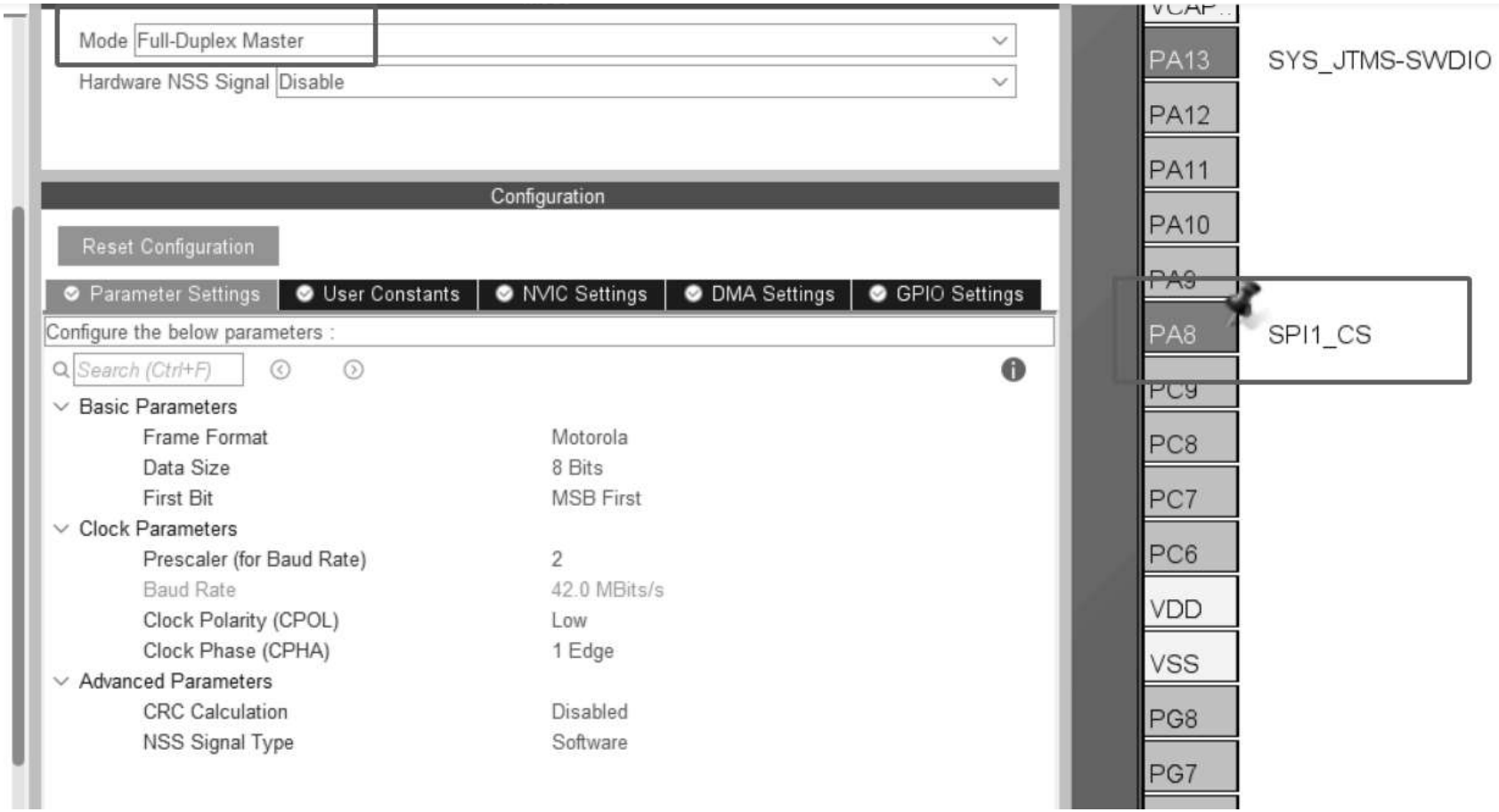

RTC Clock Mux
HSE
/ 2
HSE_RTC
Input frequency
32.768
LSE
LSE
32
To RTC (KHz)
0-1000 KHz
LSI RC
32
LSI
32
To IWDG (KHz)
32 KHz
HSI RC
16
System Clock Mux
HSI
16 MHz
HSE
SYSCLK (MHz)
AHB Prescaler
HCLK (MHz)
168
/ 1
168
168 MHz max
PLLCLK
Enable CSS
PLL Source Mux
HSI
HSE
Input frequency
8
HSE
4-26 MHz
/ 8
/ M
X 168
* N
/ 2
/ P
/ 4
/ Q
Main PLL
APB1 Prescaler
/ 4
PCLK1
42 MHz max
X 2
APB2 Prescaler
/ 2
PCLK2
84 MHz max
X 2
/ 1
168
Ethernet PTP clock (MHz)
168
HCLK to AHB bus, core, memory and DMA (MHz)
168
To Cortex System timer (MHz)
168
FCLK Cortex clock (MHz)
42
APB1 peripheral clocks (MHz)
84
APB1 Timer clocks (MHz)
84
APB2 peripheral clocks (MHz)
168
APB2 timer clocks (MHz)
84
48MHz clocks (MHz)
I2S source Mux
PLLI2SCLK
X 192
* N
/ 2
/ R
PLLI2S
PLLI2SCLK
Ext.clock
192
I2S clocks (MHz)
Input frequency
12.288
MCO2 source Mux

1ms定时器：

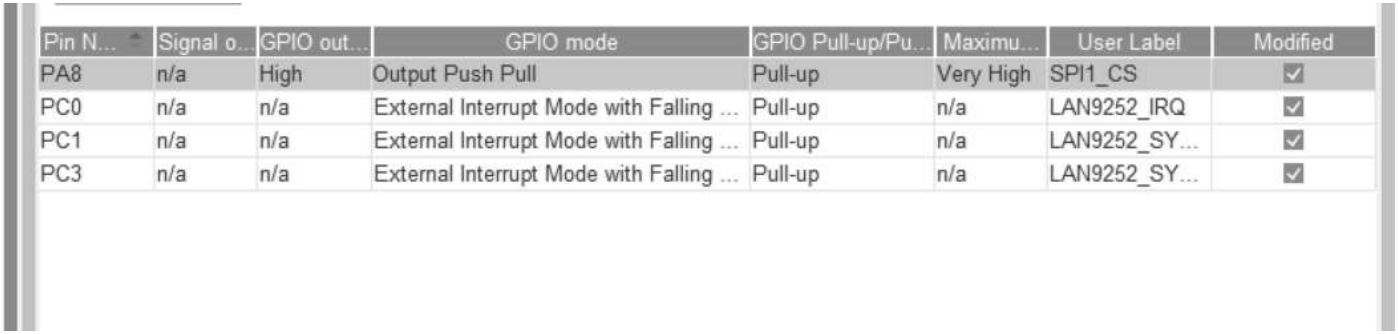




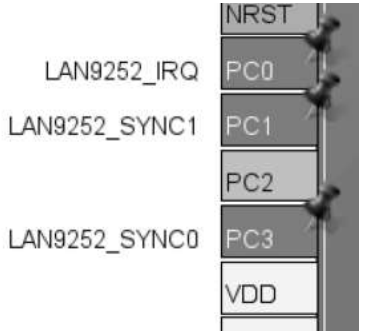
SPI配置：记得额外配置SPI_CS

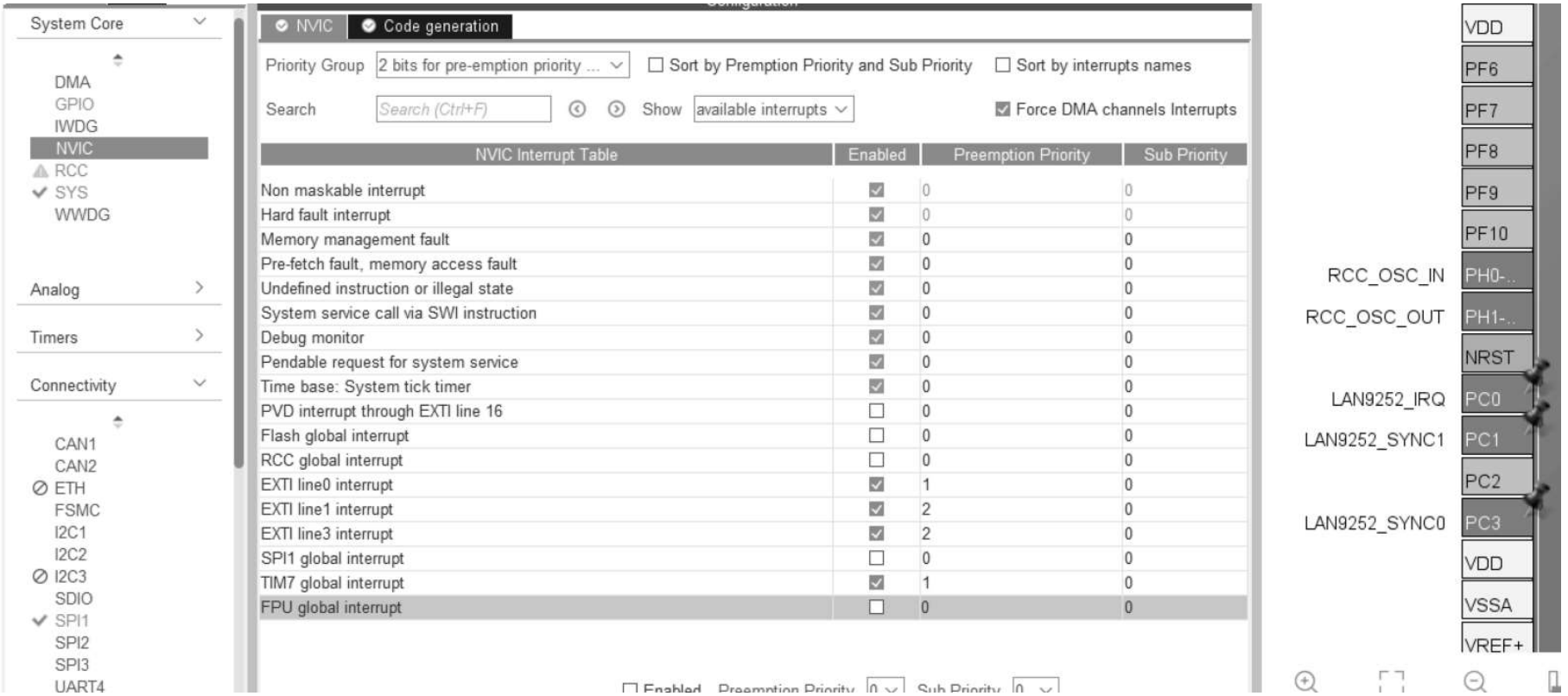


外部中断　配置：下降沿触发



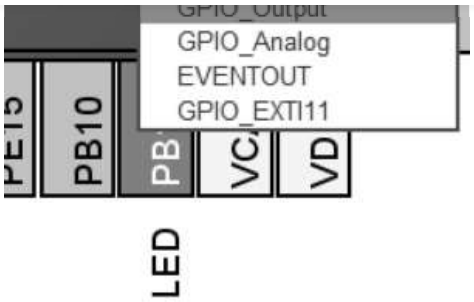
Pin N...	Signal o...	GPIO out...	GPIO mode	GPIO Pull-up/Pu...	Maximu...	User Label	Modified
PA8	n/a	High	Output Push Pull	Pull-up	Very High	SPI1_CS	☑
PC0	n/a	n/a	External Interrupt Mode with Falling ...	Pull-up	n/a	LAN9252_IRQ	☑
PC1	n/a	n/a	External Interrupt Mode with Falling ...	Pull-up	n/a	LAN9252_SY...	☑
PC3	n/a	n/a	External Interrupt Mode with Falling ...	Pull-up	n/a	LAN9252_SY...	☑



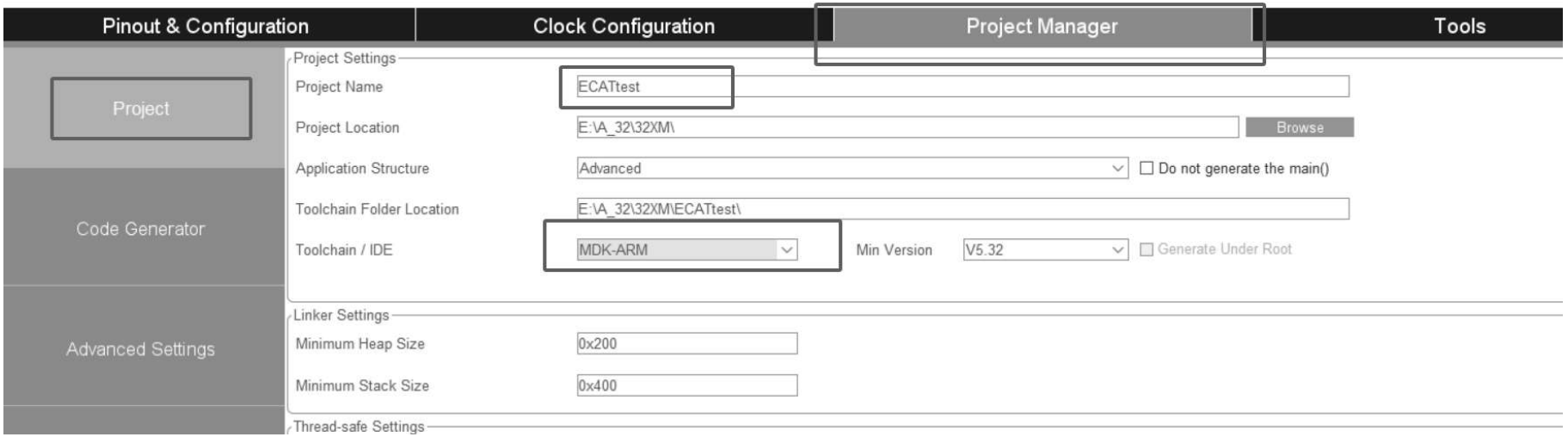


到此与LAN9252通讯需要的外设已经配置完成了，接下来可以配置其他自己所需的外设。

比如我在最一开始在字典的输出部分加入的LED来测试通讯功能。

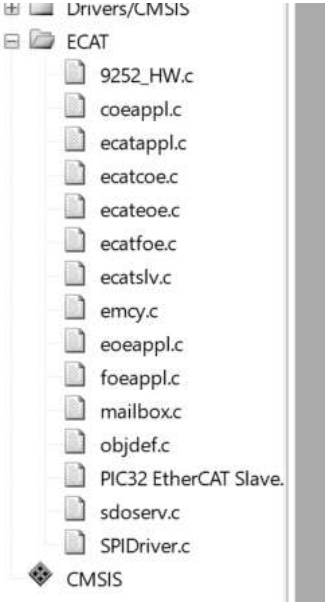

GPIO_Output
GPIO_Analog
EVENTOUT
GPIO_EXTI11
PB10
LED

创建工程：


Pinout & Configuration
Clock Configuration
Project Manager
Tools
Project
Code Generator
Advanced Settings
Project Settings
Project Name
ECATtest
Project Location
E:\A_32\32XM\
Browse
Application Structure
Advanced
Do not generate the main()
Toolchain Folder Location
E:\A_32\32XM\ECATtest\
Toolchain / IDE
MDK-ARM
Min Version
V5.32
Generate Under Root
Linker Settings
Minimum Heap Size
0x200
Minimum Stack Size
0x400
Thread-safe Settings



打开刚刚创建的工程，将刚刚生成的Src文件夹中的内容全部添加到项目，熟悉之后可以按自己的喜好和习惯分组，我这边直接全部放在了ECAT文件夹下了。



此的编译会产生一大堆报错，不要害怕，我们一点一点改掉就好了。接下来我会带你们挨个的解决这些报错，希望各位在抄答案前先能自己尝试着慢慢修改，一定能有所收货。

遇到的报错及解决方案：

我们从第一个错误位置开始依次往下解决

运行项目并下载源码 免登录复制

```
..\EtherCAT\Src\9252_HW.c(190): warning:  #223-D: function "INTDisableInterrupts" declared implicitly
        DISABLE_AL_EVENT_INT;
```

通过全局搜索定位到当前文件的106行，通过宏明可以看出这是关闭全局变量的函数，将对应位置改为用stm32的方式实现，并添加对应头文件 。

运行项目并下载源码 免登录复制

```
#include "main.h"

#define DISABLE_GLOBAL_INT          __disable_irq()
#define ENABLE_GLOBAL_INT           __enable_irq()
#define DISABLE_AL_EVENT_INT        DISABLE_GLOBAL_INT
#define ENABLE_AL_EVENT_INT         ENABLE_GLOBAL_INT
```

运行项目并下载源码 免登录复制

```
..\EtherCAT\Src\9252_HW.c(256): warning:  #223-D: function "PMPWriteDWord" declared implicitly
          PMPWriteDWord (0x54, data);
```

因为我们使用的是SPI通信，将被USE_SPI包含的252-261范围内仅保留SPIWriteDord函数，并添加SPI驱动的头文件。并对本文件内所有被#ifndef USE_SPI 过滤的代码做同样操作。

运行项目并下载源码 免登录复制

```
  #include "SPIDriver.h" //添加头文件

  //这只是部分修改后效果，未全部展示，文件内被 #ifndef USE_SPI 过滤的地方有很多，前后文都有，自己慢慢修改

    //IRQ enable,IRQ polarity, IRQ buffer type in Interrupt Configuration register.
```

```
    SPIWriteDWord (0x54, data);

    //Write in Interrupt Enable register -->
    //Write 0x5c - 0x00000001
    data = 0x00000001;

    SPIWriteDWord (0x5C, data);

    //Read Interrupt Status register
    //Read 0x58.

    SPIReadDWord(0x58);
```

往下划注释掉HW_SetLed()函数内的代码，并删除这之后的所有函数，那些是PIC32芯片的中断服务程序我们不需要。

运行项目并下载源码 免登录复制

```
void HW_SetLed(UINT8 RunLed,UINT8 ErrLed)
{
    /* Here RunLed is not used. Because on chip supported RUN Led is available*/
    //    LED_ECATRED   = ErrLed;
}
// 这个函数之下的所有内容删除
```

运行项目并下载源码 免登录复制

```
..\EtherCAT\Src\9252_HW.c(295): error:  #20: identifier "INTCONbits" is undefined
      INIT_SYNC0_INT
```

追踪到宏定义的位置，可以看出这是一个SYNC0的初始化函数，中断初始化的代码在使用Cubomx生成代码时已经自动添加了，我们直接把宏后面的定义内容删除，使其变成一个空的宏。

通过问题1和问题3不难发现9252_HW.c文件中的宏都是基于PIC32系列芯片来写的，我们只需要全部将其修改为适配STM32即可。

将114~166行所有宏定义用HAL库函数重写，并删除用不到的宏，全部修改完后如下所示：

```
#ifdef PIC32_HW
BOOL bEscInterrupt = 0;
BOOL bSync0Interrupt = 0;
BOOL bSync1Interrupt = 0;
BOOL bTimer5Interrupt = 0;
//////////////////////////////////////////////////////////////////////////
// Global Interrupt setting

#define DISABLE_GLOBAL_INT          __disable_irq()
#define ENABLE_GLOBAL_INT           __enable_irq()
#define DISABLE_AL_EVENT_INT        DISABLE_GLOBAL_INT
#define ENABLE_AL_EVENT_INT         ENABLE_GLOBAL_INT


//////////////////////////////////////////////////////////////////////////
// ESC Interrupt
    //0 - falling edge 1-
#define    INIT_ESC_INT

#define    INT_EL                   HAL_GPIO_ReadPin(LAN9252_IRQ_GPIO_Port,LAN9252_IRQ_Pin) //ESC Interrupt input port

#define    ACK_ESC_INT              __HAL_GPIO_EXTI_CLEAR_IT(LAN9252_IRQ_Pin)

#define IS_ESC_INT_ACTIVE    ((INT_EL) == 0) //0 - fro active low; 1 for hactive high
//////////////////////////////////////////////////////////////////////////
// SYNC0 Interrupt

#ifndef RUN_FROM_SVB_FPGA

    #define    INIT_SYNC0_INT

    #define    INT_SYNC0                     HAL_GPIO_ReadPin(LAN9252_SYNC0_GPIO_Port,LAN9252_SYNC0_Pin) //Sync1 Interrupt input port

    #define    DISABLE_SYNC0_INT             HAL_NVIC_DisableIRQ(LAN9252_SYNC0_EXTI_IRQn)//{(_INT1IE)=0;}//disable interrupt source INT1
    #define    ENABLE_SYNC0_INT              HAL_NVIC_EnableIRQ(LAN9252_SYNC0_EXTI_IRQn) //enable interrupt source INT1
    #define    ACK_SYNC0_INT                 __HAL_GPIO_EXTI_CLEAR_IT(LAN9252_SYNC0_Pin)


    #define    IS_SYNC0_INT_ACTIVE            ((INT_SYNC0) == 0) //0 - fro active low; 1 for hactive high

```

```
44
45      #define    DISABLE_SYNC1_INT               HAL_NVIC_DisableIRQ(LAN9252_SYNC1_EXTI_IRQn)//disable interrupt source INT2
46      #define    ENABLE_SYNC1_INT                HAL_NVIC_EnableIRQ(LAN9252_SYNC1_EXTI_IRQn) //enable interrupt source INT2
47      #define    ACK_SYNC1_INT                   __HAL_GPIO_EXTI_CLEAR_IT(LAN9252_SYNC1_Pin)
48
49
50      #define    IS_SYNC1_INT_ACTIVE             ((INT_SYNC1) == 0) //0 - fro active low; 1 for hactive high
51  #else
52
53      // Place-holder
54
55  #endif
56
57  ////////////////////////////////////////////////////////////////////////
58  // Hardware timer
59
60  #define STOP_ECAT_TIMER             HAL_TIM_Base_Stop_IT(&htim7)
61  #define INIT_ECAT_TIMER             HAL_TIM_Base_Init(&htim7)
62
63  #define START_ECAT_TIMER          HAL_TIM_Base_Start_IT(&htim7)
64
65  #endif // end of PIC32_HW
```

改完之后重新编译会发现少了很多错误，接着一个一个改就好了。

运行项目并下载源码　　免登录复制

```
1  ..\EtherCAT\Src\ecatappl.c(250): warning:  #223-D: function "HW_GetTimer" declared implicitly
2                StartTimerCnt = (UINT32) HW_GetTimer();
```

跟第一个问题同样的方法，找到宏所在位置，发现是灰色的没被编译，可以选择将限制条件#ifdef PIC32_HW 删掉，也可以选择改为#ifdef STM32F4（具体定义什么根据自己的芯片型号来，需要头文件main.h）注释掉两个没有的头文件和 “ // Defines，types ”区域内的所有内容。

将#define ECAT_TIMER_INC_P_MS 后面的数值改为你配置1ms定时器时的计数重载器的值。

再根据宏名和给的注释将152-166行附近的宏以同样的方式用HAL库函数实现。

修改后代码如下：

```
//添加头文件
#include "main.h"
#include "tim.h"
#include "spi.h"
#include "gpio.h"
#include "stm32f4xx_hal.h"

/////////////////////////////////////////////////////////////////////////
// Hardware timer settings

#define ECAT_TIMER_INC_P_MS              2000 /**< \brief 312 ticks per ms*/


/////////////////////////////////////////////////////////////////////////
// Interrupt and Timer Defines

#ifndef DISABLE_ESC_INT
    #define    DISABLE_ESC_INT()            HAL_NVIC_DisableIRQ(LAN9252_SYNC0_EXTI_IRQn) /**< \brief Disable interrupt source INT1*/
#endif
#ifndef ENABLE_ESC_INT
    #define    ENABLE_ESC_INT()             HAL_NVIC_EnableIRQ(LAN9252_SYNC0_EXTI_IRQn) /**< \brief Enable interrupt source INT1*/
#endif

//TODO
#ifndef HW_GetTimer
    #define HW_GetTimer()           __HAL_TIM_GET_COUNTER(&htim7) /**< \brief Access to the hardware timer*/
#endif

#ifndef HW_ClearTimer
    #define HW_ClearTimer()       __HAL_TIM_SET_COUNTER(&htim7,0) /**< \brief Clear the hardware timer*/
#endif

#endif
```

改完再重新编译一下又少了很多错误，此时第一个错误应该是来自 SPIDriver.h 文件，接下来借此机会我们来直接修改SPI驱动使适配STM32。

在 SPIDriver.h 中修改如下内容：

删除plib.h,并增加如下头文件

```
#include "ecat_def.h"
#include "gpio.h"
#include "spi.h"
#include "main.h"
```

将宏CSLOW()与CSHIGH()修复为自己SPI的控制引脚

运行项目并下载源码 免登录复制

```
    #define SPI_OFF             HAL_GPIO_WritePin(SPI1_CS_GPIO_Port,SPI1_CS_Pin,GPIO_PIN_SET)
    #define SPI_ON              HAL_GPIO_WritePin(SPI1_CS_GPIO_Port,SPI1_CS_Pin,GPIO_PIN_RESET)

    #define CSLOW()         SPI_ON
    #define CSHIGH()        SPI_OFF
```

在 SPIDriver.c 中修改如下内容：

删除Delay、SPIPut、SPIOpen三个函数。

修改SPIWrite与SPIRead函数，用HAL库方式实现。

运行项目并下载源码 免登录复制

```
void SPIWrite(UINT8 data)
{
    HAL_SPI_Transmit(&hspi1,&data,1,2000);
}
UINT8 SPIRead()
{
    UINT8 data;
    HAL_SPI_Receive(&hspi1,&data,1,2000);
    return (data);
}
```

剩下的就是UINT32_VAL与UINT16_VAL类型未定义引起的问题，这两个变量是PIC芯片内的库文件包含的，我们添加一个 "GenericTypeDefs.h" 头文件到工程并引用即可。

GenericTypeDefs.h

```
/*********************************************************************

                  Generic Type Definitions

*********************************************************************
 FileName:        GenericTypeDefs.h
 Dependencies:    None
 Processor:       PIC10, PIC12, PIC16, PIC18, PIC24, dsPIC, PIC32
 Compiler:        MPLAB C Compilers for PIC18, PIC24, dsPIC, & PIC32
                  Hi-Tech PICC PRO, Hi-Tech PICC18 PRO
 Company:         Microchip Technology Inc.

 Software License Agreement

 The software supplied herewith by Microchip Technology Incorporated
 (the "Company") is intended and supplied to you, the Company's
 customer, for use solely and exclusively with products manufactured
 by the Company.

 The software is owned by the Company and/or its supplier, and is
 protected under applicable copyright laws. All rights are reserved.
 Any use in violation of the foregoing restrictions may subject the
 user to criminal sanctions under applicable laws, as well as to
 civil liability for the breach of the terms and conditions of this
 license.

 THIS SOFTWARE IS PROVIDED IN AN "AS IS" CONDITION. NO WARRANTIES,
 WHETHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, BUT NOT LIMITED
 TO, IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
 PARTICULAR PURPOSE APPLY TO THIS SOFTWARE. THE COMPANY SHALL NOT,
 IN ANY CIRCUMSTANCES, BE LIABLE FOR SPECIAL, INCIDENTAL OR
 CONSEQUENTIAL DAMAGES, FOR ANY REASON WHATSOEVER.

*********************************************************************
 File Description:

 Change History:
  Rev   Date         Description
  1.1   09/11/06     Add base signed types
  1.2   02/28/07     Add QWORD, LONGLONG, QWORD_VAL
```

```
  Draft 2.0   07/13/09     Updated for new release of coding standards
*****************************************************************/

#ifndef __GENERIC_TYPE_DEFS_H_
#define __GENERIC_TYPE_DEFS_H_

#ifdef __cplusplus
extern "C"
  {
#endif

/* Specify an extension for GCC based compilers */
#if defined(__GNUC__)
#define __EXTENSION __extension__
#else
#define __EXTENSION
#endif

/* get compiler defined type definitions (NULL, size_t, etc) */
#include <stddef.h>
#include "ecat_def.h"

//typedef enum _BOOL { FALSE = 0, TRUE } BOOL;    /* Undefined size */
//typedef enum _BIT { CLEAR = 0, SET } BIT;

#define PUBLIC                                  /* Function attributes */
#define PROTECTED
#define PRIVATE   static

/* INT is processor specific in length may vary in size */
//typedef signed int          INT;
//typedef signed char         INT8;
//typedef signed short int    INT16;
//typedef signed long int     INT32;

/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
#if !defined(__18CXX)
__EXTENSION typedef signed long long    INT64;
#endif
```

```
//typedef unsigned char       UINT8;
//typedef unsigned short int  UINT16;
/* 24-bit type only available on C18 */
#if defined(__18CXX)
typedef unsigned short long UINT24;
#endif
//typedef unsigned long int   UINT32;     /* other name for 32-bit integer */
/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
#if !defined(__18CXX)
__EXTENSION typedef unsigned long long  UINT64;
#endif

typedef union
{
    UINT8 Val;
    struct
    {
        __EXTENSION UINT8 b0:1;
        __EXTENSION UINT8 b1:1;
        __EXTENSION UINT8 b2:1;
        __EXTENSION UINT8 b3:1;
        __EXTENSION UINT8 b4:1;
        __EXTENSION UINT8 b5:1;
        __EXTENSION UINT8 b6:1;
        __EXTENSION UINT8 b7:1;
    } bits;
} UINT8_VAL, UINT8_BITS;

typedef union
{
    UINT16 Val;
    UINT8 v[2];
    struct
    {
        UINT8 LB;
        UINT8 HB;
    } byte;
    struct
    {
        __EXTENSION UINT8 b0:1;
```

```
        __EXTENSION UINT8 b3:1;
        __EXTENSION UINT8 b4:1;
        __EXTENSION UINT8 b5:1;
        __EXTENSION UINT8 b6:1;
        __EXTENSION UINT8 b7:1;
        __EXTENSION UINT8 b8:1;
        __EXTENSION UINT8 b9:1;
        __EXTENSION UINT8 b10:1;
        __EXTENSION UINT8 b11:1;
        __EXTENSION UINT8 b12:1;
        __EXTENSION UINT8 b13:1;
        __EXTENSION UINT8 b14:1;
        __EXTENSION UINT8 b15:1;
    } bits;
} UINT16_VAL, UINT16_BITS;

/* 24-bit type only available on C18 */
#if defined(__18CXX)
typedef union
{
    UINT24 Val;
    UINT8 v[3];
    struct
    {
        UINT8 LB;
        UINT8 HB;
        UINT8 UB;
    } byte;
    struct
    {
        __EXTENSION UINT8 b0:1;
        __EXTENSION UINT8 b1:1;
        __EXTENSION UINT8 b2:1;
        __EXTENSION UINT8 b3:1;
        __EXTENSION UINT8 b4:1;
        __EXTENSION UINT8 b5:1;
        __EXTENSION UINT8 b6:1;
        __EXTENSION UINT8 b7:1;
        __EXTENSION UINT8 b8:1;
        __EXTENSION UINT8 b9:1;
```

```
        __EXTENSION UINT8 b13:1;
        __EXTENSION UINT8 b14:1;
        __EXTENSION UINT8 b15:1;
        __EXTENSION UINT8 b16:1;
        __EXTENSION UINT8 b17:1;
        __EXTENSION UINT8 b18:1;
        __EXTENSION UINT8 b19:1;
        __EXTENSION UINT8 b20:1;
        __EXTENSION UINT8 b21:1;
        __EXTENSION UINT8 b22:1;
        __EXTENSION UINT8 b23:1;
    } bits;
} UINT24_VAL, UINT24_BITS;
#endif

typedef union
{
    UINT32 Val;
    UINT16 w[2];
    UINT8  v[4];
    struct
    {
        UINT16 LW;
        UINT16 HW;
    } word;
    struct
    {
        UINT8 LB;
        UINT8 HB;
        UINT8 UB;
        UINT8 MB;
    } byte;
    struct
    {
        UINT16_VAL low;
        UINT16_VAL high;
    }wordUnion;
    struct
    {
        __EXTENSION UINT8 b0:1;
```

```
        __EXTENSION UINT8 b3:1;
        __EXTENSION UINT8 b4:1;
        __EXTENSION UINT8 b5:1;
        __EXTENSION UINT8 b6:1;
        __EXTENSION UINT8 b7:1;
        __EXTENSION UINT8 b8:1;
        __EXTENSION UINT8 b9:1;
        __EXTENSION UINT8 b10:1;
        __EXTENSION UINT8 b11:1;
        __EXTENSION UINT8 b12:1;
        __EXTENSION UINT8 b13:1;
        __EXTENSION UINT8 b14:1;
        __EXTENSION UINT8 b15:1;
        __EXTENSION UINT8 b16:1;
        __EXTENSION UINT8 b17:1;
        __EXTENSION UINT8 b18:1;
        __EXTENSION UINT8 b19:1;
        __EXTENSION UINT8 b20:1;
        __EXTENSION UINT8 b21:1;
        __EXTENSION UINT8 b22:1;
        __EXTENSION UINT8 b23:1;
        __EXTENSION UINT8 b24:1;
        __EXTENSION UINT8 b25:1;
        __EXTENSION UINT8 b26:1;
        __EXTENSION UINT8 b27:1;
        __EXTENSION UINT8 b28:1;
        __EXTENSION UINT8 b29:1;
        __EXTENSION UINT8 b30:1;
        __EXTENSION UINT8 b31:1;
    } bits;
} UINT32_VAL;

/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
#if !defined(__18CXX)
typedef union
{
    UINT64 Val;
    UINT32 d[2];
    UINT16 w[4];
    UINT8 v[8];
```

```
        UINT32 HD;
    } dword;
    struct
    {
        UINT16 LW;
        UINT16 HW;
        UINT16 UW;
        UINT16 MW;
    } word;
    struct
    {
        __EXTENSION UINT8 b0:1;
        __EXTENSION UINT8 b1:1;
        __EXTENSION UINT8 b2:1;
        __EXTENSION UINT8 b3:1;
        __EXTENSION UINT8 b4:1;
        __EXTENSION UINT8 b5:1;
        __EXTENSION UINT8 b6:1;
        __EXTENSION UINT8 b7:1;
        __EXTENSION UINT8 b8:1;
        __EXTENSION UINT8 b9:1;
        __EXTENSION UINT8 b10:1;
        __EXTENSION UINT8 b11:1;
        __EXTENSION UINT8 b12:1;
        __EXTENSION UINT8 b13:1;
        __EXTENSION UINT8 b14:1;
        __EXTENSION UINT8 b15:1;
        __EXTENSION UINT8 b16:1;
        __EXTENSION UINT8 b17:1;
        __EXTENSION UINT8 b18:1;
        __EXTENSION UINT8 b19:1;
        __EXTENSION UINT8 b20:1;
        __EXTENSION UINT8 b21:1;
        __EXTENSION UINT8 b22:1;
        __EXTENSION UINT8 b23:1;
        __EXTENSION UINT8 b24:1;
        __EXTENSION UINT8 b25:1;
        __EXTENSION UINT8 b26:1;
        __EXTENSION UINT8 b27:1;
```

```
        __EXTENSION UINT8 b31:1;
        __EXTENSION UINT8 b32:1;
        __EXTENSION UINT8 b33:1;
        __EXTENSION UINT8 b34:1;
        __EXTENSION UINT8 b35:1;
        __EXTENSION UINT8 b36:1;
        __EXTENSION UINT8 b37:1;
        __EXTENSION UINT8 b38:1;
        __EXTENSION UINT8 b39:1;
        __EXTENSION UINT8 b40:1;
        __EXTENSION UINT8 b41:1;
        __EXTENSION UINT8 b42:1;
        __EXTENSION UINT8 b43:1;
        __EXTENSION UINT8 b44:1;
        __EXTENSION UINT8 b45:1;
        __EXTENSION UINT8 b46:1;
        __EXTENSION UINT8 b47:1;
        __EXTENSION UINT8 b48:1;
        __EXTENSION UINT8 b49:1;
        __EXTENSION UINT8 b50:1;
        __EXTENSION UINT8 b51:1;
        __EXTENSION UINT8 b52:1;
        __EXTENSION UINT8 b53:1;
        __EXTENSION UINT8 b54:1;
        __EXTENSION UINT8 b55:1;
        __EXTENSION UINT8 b56:1;
        __EXTENSION UINT8 b57:1;
        __EXTENSION UINT8 b58:1;
        __EXTENSION UINT8 b59:1;
        __EXTENSION UINT8 b60:1;
        __EXTENSION UINT8 b61:1;
        __EXTENSION UINT8 b62:1;
        __EXTENSION UINT8 b63:1;
    } bits;
} UINT64_VAL;
#endif /* __18CXX */

/****************************************************************************/

/* Alternate definitions */
```

```
typedef char                    CHAR8;
typedef unsigned char           UCHAR8;

typedef unsigned char           BYTE;                       /* 8-bit unsigned  */
typedef unsigned short int      WORD;                       /* 16-bit unsigned */
typedef unsigned long           DWORD;                      /* 32-bit unsigned */
/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
#if !defined(__18CXX)
__EXTENSION
typedef unsigned long long      QWORD;                      /* 64-bit unsigned */
#endif /* __18CXX */
//typedef signed char             CHAR;                       /* 8-bit signed    */
typedef signed short int        SHORT;                      /* 16-bit signed   */
typedef signed long             LONG;                       /* 32-bit signed   */
/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
#if !defined(__18CXX)
__EXTENSION
typedef signed long long        LONGLONG;                   /* 64-bit signed   */
#endif /* __18CXX */
typedef union
{
    BYTE Val;
    struct
    {
        __EXTENSION BYTE b0:1;
        __EXTENSION BYTE b1:1;
        __EXTENSION BYTE b2:1;
        __EXTENSION BYTE b3:1;
        __EXTENSION BYTE b4:1;
        __EXTENSION BYTE b5:1;
        __EXTENSION BYTE b6:1;
        __EXTENSION BYTE b7:1;
    } bits;
} BYTE_VAL, BYTE_BITS;

typedef union
{
    WORD Val;
    BYTE v[2];
    struct
```

```
    } byte;
    struct
    {
        __EXTENSION BYTE b0:1;
        __EXTENSION BYTE b1:1;
        __EXTENSION BYTE b2:1;
        __EXTENSION BYTE b3:1;
        __EXTENSION BYTE b4:1;
        __EXTENSION BYTE b5:1;
        __EXTENSION BYTE b6:1;
        __EXTENSION BYTE b7:1;
        __EXTENSION BYTE b8:1;
        __EXTENSION BYTE b9:1;
        __EXTENSION BYTE b10:1;
        __EXTENSION BYTE b11:1;
        __EXTENSION BYTE b12:1;
        __EXTENSION BYTE b13:1;
        __EXTENSION BYTE b14:1;
        __EXTENSION BYTE b15:1;
    } bits;
} WORD_VAL, WORD_BITS;

typedef union
{
    DWORD Val;
    WORD w[2];
    BYTE v[4];
    struct
    {
        WORD LW;
        WORD HW;
    } word;
    struct
    {
        BYTE LB;
        BYTE HB;
        BYTE UB;
        BYTE MB;
    } byte;
    struct
```

```
        WORD_VAL high;
    }wordUnion;
    struct
    {
        __EXTENSION BYTE b0:1;
        __EXTENSION BYTE b1:1;
        __EXTENSION BYTE b2:1;
        __EXTENSION BYTE b3:1;
        __EXTENSION BYTE b4:1;
        __EXTENSION BYTE b5:1;
        __EXTENSION BYTE b6:1;
        __EXTENSION BYTE b7:1;
        __EXTENSION BYTE b8:1;
        __EXTENSION BYTE b9:1;
        __EXTENSION BYTE b10:1;
        __EXTENSION BYTE b11:1;
        __EXTENSION BYTE b12:1;
        __EXTENSION BYTE b13:1;
        __EXTENSION BYTE b14:1;
        __EXTENSION BYTE b15:1;
        __EXTENSION BYTE b16:1;
        __EXTENSION BYTE b17:1;
        __EXTENSION BYTE b18:1;
        __EXTENSION BYTE b19:1;
        __EXTENSION BYTE b20:1;
        __EXTENSION BYTE b21:1;
        __EXTENSION BYTE b22:1;
        __EXTENSION BYTE b23:1;
        __EXTENSION BYTE b24:1;
        __EXTENSION BYTE b25:1;
        __EXTENSION BYTE b26:1;
        __EXTENSION BYTE b27:1;
        __EXTENSION BYTE b28:1;
        __EXTENSION BYTE b29:1;
        __EXTENSION BYTE b30:1;
        __EXTENSION BYTE b31:1;
    } bits;
} DWORD_VAL;

/* MPLAB C Compiler for PIC18 does not support 64-bit integers */
```

```
{
    QWORD Val;
    DWORD d[2];
    WORD w[4];
    BYTE v[8];
    struct
    {
        DWORD LD;
        DWORD HD;
    } dword;
    struct
    {
        WORD LW;
        WORD HW;
        WORD UW;
        WORD MW;
    } word;
    struct
    {
        __EXTENSION BYTE b0:1;
        __EXTENSION BYTE b1:1;
        __EXTENSION BYTE b2:1;
        __EXTENSION BYTE b3:1;
        __EXTENSION BYTE b4:1;
        __EXTENSION BYTE b5:1;
        __EXTENSION BYTE b6:1;
        __EXTENSION BYTE b7:1;
        __EXTENSION BYTE b8:1;
        __EXTENSION BYTE b9:1;
        __EXTENSION BYTE b10:1;
        __EXTENSION BYTE b11:1;
        __EXTENSION BYTE b12:1;
        __EXTENSION BYTE b13:1;
        __EXTENSION BYTE b14:1;
        __EXTENSION BYTE b15:1;
        __EXTENSION BYTE b16:1;
        __EXTENSION BYTE b17:1;
        __EXTENSION BYTE b18:1;
        __EXTENSION BYTE b19:1;
        __EXTENSION BYTE b20:1;
```

```
510         __EXTENSION BYTE b24:1;
511         __EXTENSION BYTE b25:1;
512         __EXTENSION BYTE b26:1;
513         __EXTENSION BYTE b27:1;
514         __EXTENSION BYTE b28:1;
515         __EXTENSION BYTE b29:1;
516         __EXTENSION BYTE b30:1;
517         __EXTENSION BYTE b31:1;
518         __EXTENSION BYTE b32:1;
519         __EXTENSION BYTE b33:1;
520         __EXTENSION BYTE b34:1;
521         __EXTENSION BYTE b35:1;
522         __EXTENSION BYTE b36:1;
523         __EXTENSION BYTE b37:1;
524         __EXTENSION BYTE b38:1;
525         __EXTENSION BYTE b39:1;
526         __EXTENSION BYTE b40:1;
527         __EXTENSION BYTE b41:1;
528         __EXTENSION BYTE b42:1;
529         __EXTENSION BYTE b43:1;
530         __EXTENSION BYTE b44:1;
531         __EXTENSION BYTE b45:1;
532         __EXTENSION BYTE b46:1;
533         __EXTENSION BYTE b47:1;
534         __EXTENSION BYTE b48:1;
535         __EXTENSION BYTE b49:1;
536         __EXTENSION BYTE b50:1;
537         __EXTENSION BYTE b51:1;
538         __EXTENSION BYTE b52:1;
539         __EXTENSION BYTE b53:1;
540         __EXTENSION BYTE b54:1;
541         __EXTENSION BYTE b55:1;
542         __EXTENSION BYTE b56:1;
543         __EXTENSION BYTE b57:1;
544         __EXTENSION BYTE b58:1;
545         __EXTENSION BYTE b59:1;
546         __EXTENSION BYTE b60:1;
547         __EXTENSION BYTE b61:1;
548         __EXTENSION BYTE b62:1;
549         __EXTENSION BYTE b63:1;
```

```
#endif /* __18CXX */

#undef __EXTENSION

#ifdef __cplusplus
  }
#endif
#endif /* __GENERIC_TYPE_DEFS_H_ */
```

问题：

运行项目并下载源码 免登录复制

```
..\EtherCAT\Src\9252_HW.c(316): warning:  #223-D: function "ConfigIntTimer5" declared implicitly
      ConfigIntTimer5(T5_INT_ON | T5_INT_PRIOR_3 );
```

没有定义的函数，直接删除。

问题：

运行项目并下载源码 免登录复制

```
..\EtherCAT\Inc\eoeappl.h(209): error:  #1032: Definition of nested anonymous union in packed "struct <unnamed>" must be __packed
      } u; /**< \brief Data*/
```

这个错误是由 __packed 关键字引发的，我们直接使用Ctrl+F全局搜索关键字packed，追踪到了ecat_def.h的742行和754行，将这两个宏后面的映射语句删掉，使其变成空的宏。

修改完这处错误不出意外的话再次编译就不会报错了，剩下的一些警告可以自己解决，当然，直接无视也没什么问题。

接下来就是将协议栈暴露给我们的几个函数在代码相应的位置调用，以及编写自己的业务逻辑。

1.在 LAN9252_IRQ引脚对应的中断服务程序中调用 **PDI_Isr()**;我这里对应的是EXTI0

运行项目并下载源码 免登录复制

```
#include "applInterface.h" //添加头文件

void EXTI0_IRQHandler(void)
```

```
  /* USER CODE END EXTI0_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(LAN9252_IRQ_Pin);
  /* USER CODE BEGIN EXTI0_IRQn 1 */

  /* USER CODE END EXTI0_IRQn 1 */
}
```

2.在 LAN9252_SUNC0和LAN9252_SUNC1引脚对应的中断服务程序中调用 **Sync0_Isr() 和 Sync1_Isr()**

运行项目并下载源码 免登录复制

```
/**
  * @brief This function handles EXTI line1 interrupt.
  */
void EXTI1_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI1_IRQn 0 */
    DISABLE_ESC_INT();
    Sync1_Isr();
    ENABLE_ESC_INT();
  /* USER CODE END EXTI1_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(LAN9252_SYNC1_Pin);
  /* USER CODE BEGIN EXTI1_IRQn 1 */

  /* USER CODE END EXTI1_IRQn 1 */
}

/**
  * @brief This function handles EXTI line3 interrupt.
  */
void EXTI3_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI3_IRQn 0 */
    DISABLE_ESC_INT();
    Sync0_Isr();
    ENABLE_ESC_INT();
  /* USER CODE END EXTI3_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(LAN9252_SYNC0_Pin);
  /* USER CODE BEGIN EXTI3_IRQn 1 */
```

```
}
```

注意，设备上电后我们不可以直接调用HAL_NVIC_EnableIRQ函数将各种中断使能，因为此时协议栈相关数据都没初始化好，直接使能这些中断将导致严重的逻辑问题。

因此我们需要回到gpio.c中，将CuboMX自动生成的使能语句注释掉。

运行项目并下载源码 免登录复制

```
  位置在MX_GPIO_Init()函数的结尾。

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI0_IRQn, 1, 0);
  // HAL_NVIC_EnableIRQ(EXTI0_IRQn);

  HAL_NVIC_SetPriority(EXTI1_IRQn, 2, 0);
  // HAL_NVIC_EnableIRQ(EXTI1_IRQn);

  HAL_NVIC_SetPriority(EXTI3_IRQn, 2, 0);
  // HAL_NVIC_EnableIRQ(EXTI3_IRQn);
```

3. 在自己设置的1ms定时器的中断服务函数中调用ECAT_CheckTimer()

运行项目并下载源码 免登录复制

```
void TIM7_IRQHandler(void)
{
  /* USER CODE BEGIN TIM7_IRQn 0 */
    ECAT_CheckTimer();
  /* USER CODE END TIM7_IRQn 0 */
  HAL_TIM_IRQHandler(&htim7);
  /* USER CODE BEGIN TIM7_IRQn 1 */

  /* USER CODE END TIM7_IRQn 1 */
}
```

4. 在主函数中添加头文件 #include "applInterface.h" ，调用协议栈的初始化函数HW_Init()和MainInit()，并在while循环中调用MainLoop。

运行项目并下载源码 免登录复制

```
/* USER CODE BEGIN 2 */
   HW_Init();
   MainInit();
 /* USER CODE END 2 */

 /* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
   /* USER CODE END WHILE */

   /* USER CODE BEGIN 3 */
       MainLoop();
 }
```

5. 完成上述步骤剩下的就是在协议栈开放给我们的业务逻辑接口中编辑自己的业务逻辑。

如果你们一开始使用SSC生成协议栈时没自己命名的话，接口可以在PIC32 EtherCAT Slave.c文件中找到。

需要自己写的主要是下面三个函数：

运行项目并下载源码 免登录复制

```
//输入数据映射，将需要发送的数据映射到pData指向的空间
void APPL_InputMapping(UINT16* pData)

//输出数据映射，将想要接收的数据从pData指向的空间取出
void APPL_OutputMapping(UINT16* pData)

//应用逻辑
void APPL_Application(void)
```

下面是我的一段参考代码，变量Value0x6000与Led0x7000是与自己当时在Excel中添加的字典是一一对应的，变量名可以在 PIC32 EtherCAT SlaveQbjects.h 文件中找到，以后如果需要修改对象字典只需要使用SSC重新生成协议栈文件将后，把工程原有的PIC32 EtherCAT SlaveQbjects.h 替换成新生成的，并重写这三个函数即可。

```
/////////////////////////////////////////////////////////////////////////////////////////
/**
\param      pData  pointer to input process data

\brief      This function will copies the inputs from the local memory to the ESC memory
            to the hardware
*/////////////////////////////////////////////////////////////////////////////////////////
void APPL_InputMapping(UINT16* pData)
{
    pData[0] = Values0x6000.V1;
    pData[1] = Values0x6000.V2;
}
;
/////////////////////////////////////////////////////////////////////////////////////////
/**
\param      pData  pointer to output process data

\brief    This function will copies the outputs from the ESC memory to the local memory
            to the hardware
*/////////////////////////////////////////////////////////////////////////////////////////
void APPL_OutputMapping(UINT16* pData)
{
    Led0x7000.D1 = *pData;
}

/////////////////////////////////////////////////////////////////////////////////////////
/**
\brief    This function will called from the synchronisation ISR
            or from the mainloop if no synchronisation is supported
*/////////////////////////////////////////////////////////////////////////////////////////
void APPL_Application(void)
{
    Values0x6000.V1 = 0x1234;
    Values0x6000.V2 = 0x5678;

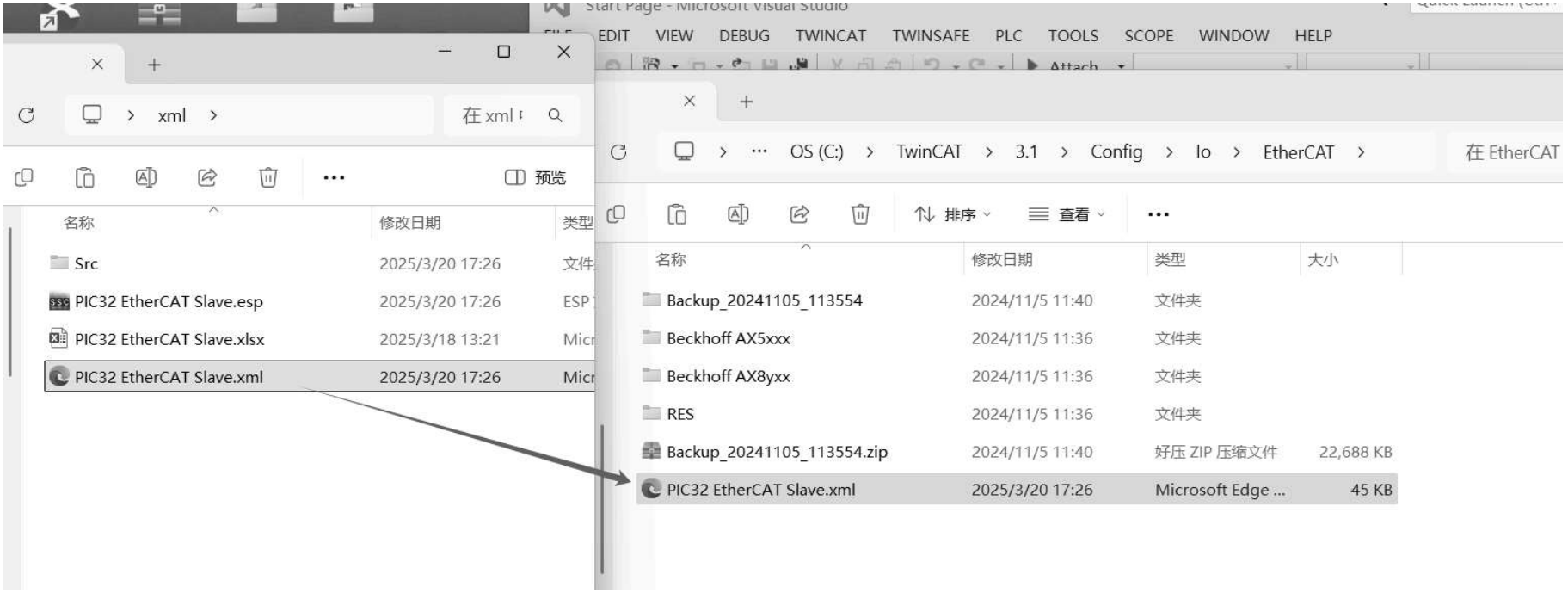
    Led0x7000.D1 ? HAL_GPIO_WritePin(LED_GPIO_Port,LED_Pin,GPIO_PIN_SET) : HAL_GPIO_WritePin(LED_GPIO_Port,LED_Pin,GPIO_PIN_RESET);

}
```

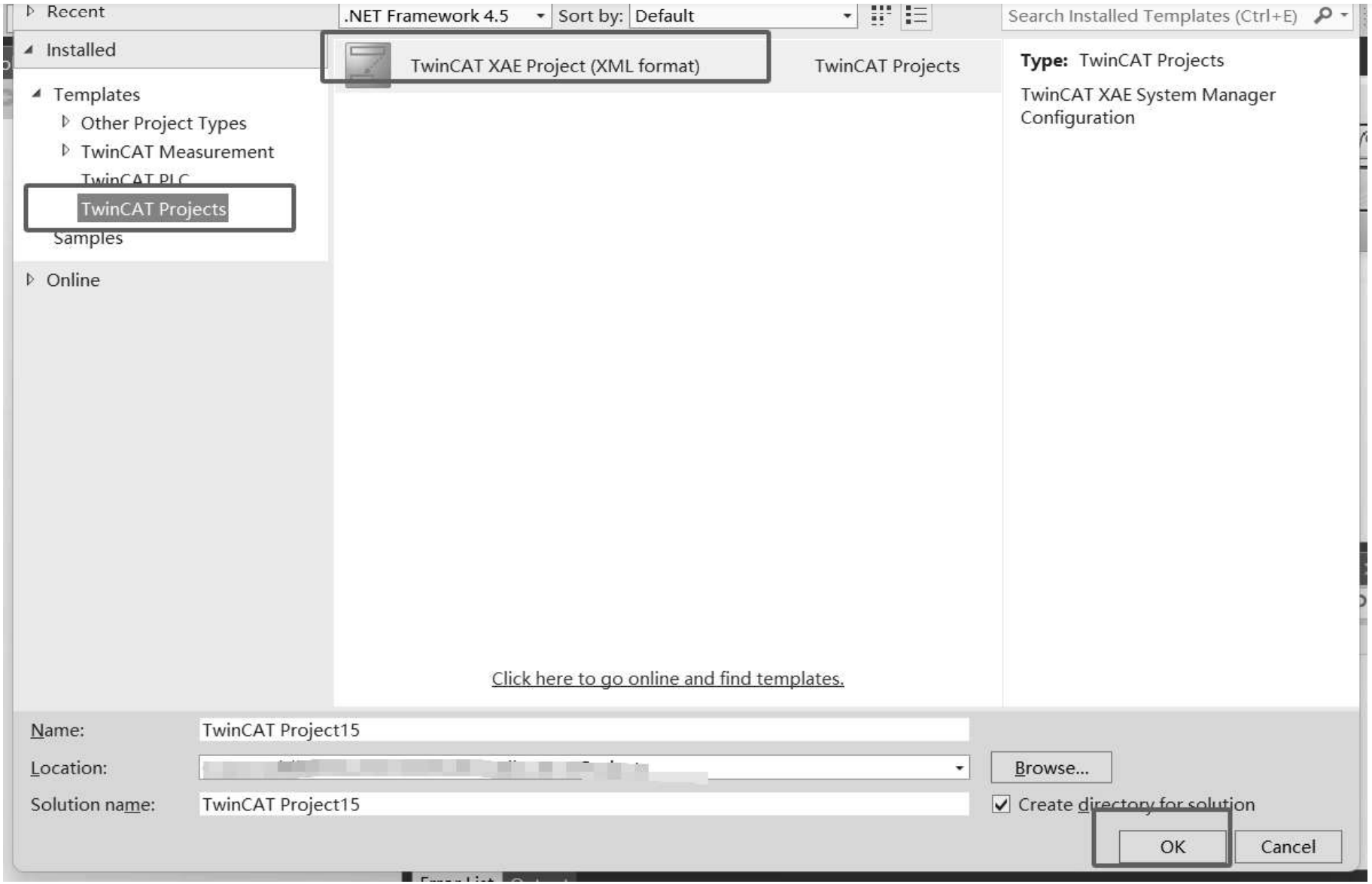
四、测试

我这里使用的是 TwinCAT3，依然是自己找资源下载即可。

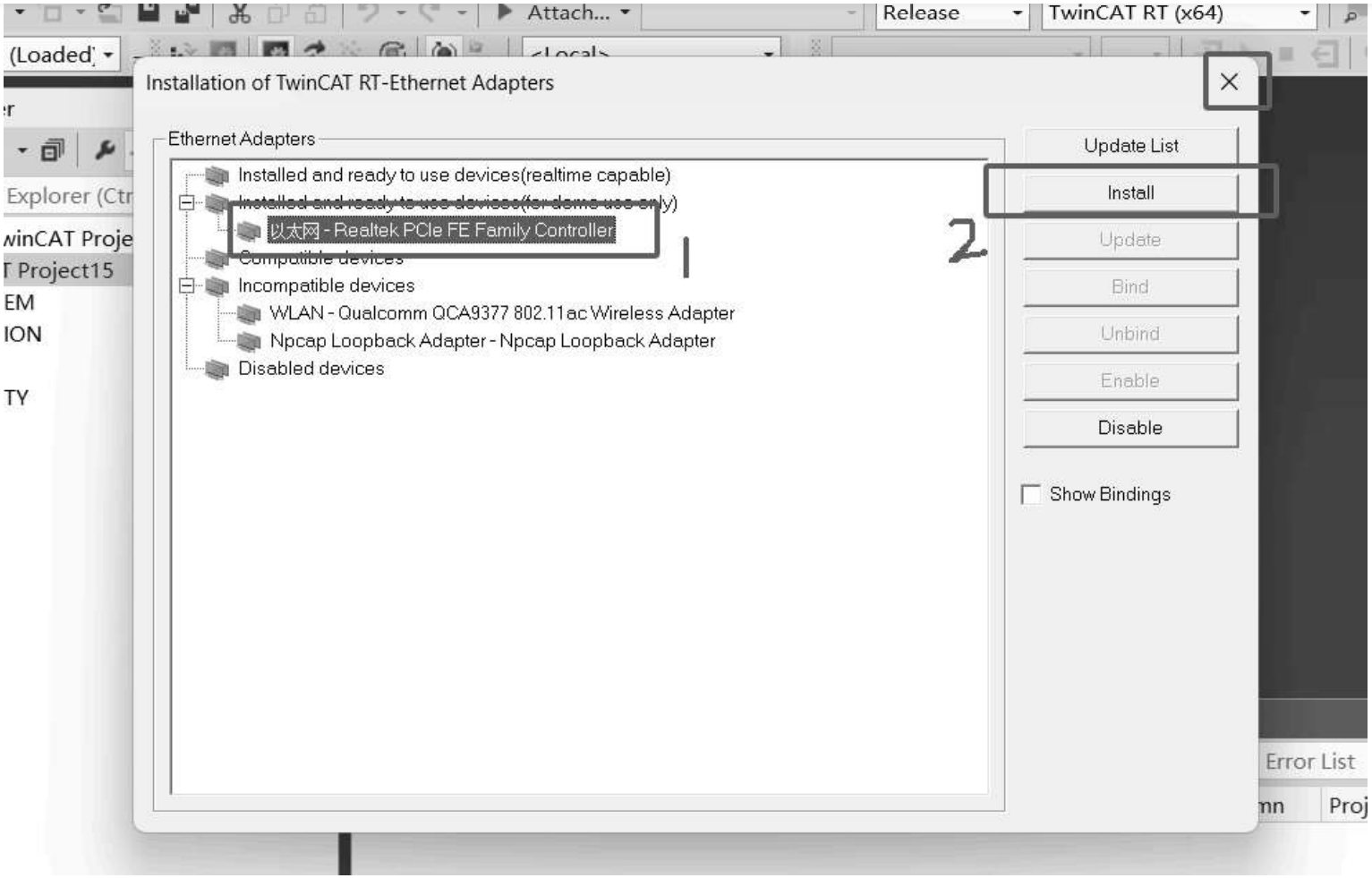
1.将 C:\TwinCAT\3.1\Config\Io\EtherCAT 路径下的所有 .xml 文件 删除，并将SSC生成的XML文件拷贝过来。



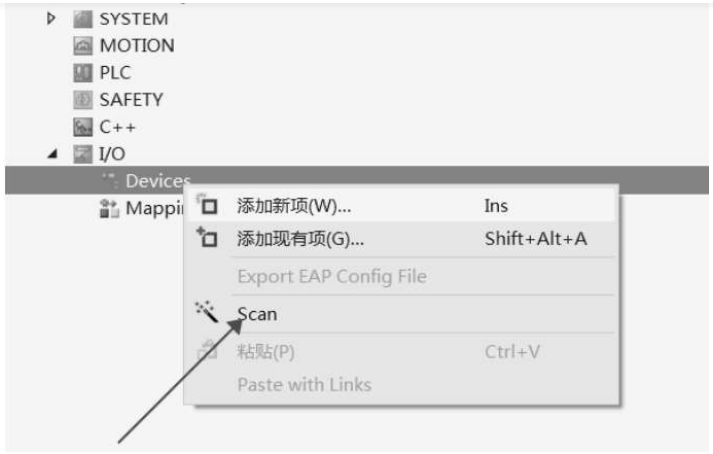
2.打开TwinCAT3新建一个EtherCAT工程，FILE -> New -> Project

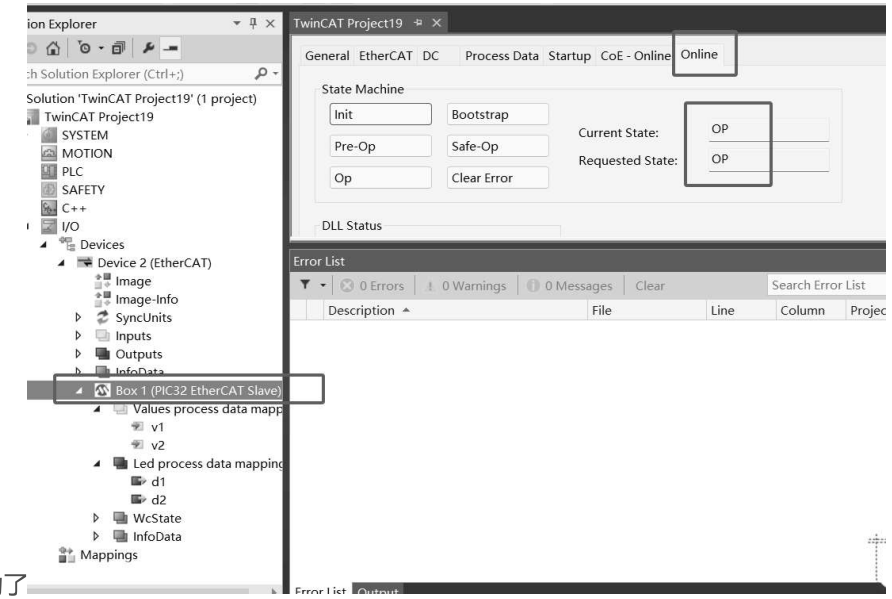


3. 先用一根网线将你的电路板与电脑连接起来，然后添加网卡驱动 TWNCAT -> Show Realtime EtherCAT Compatible Devices...



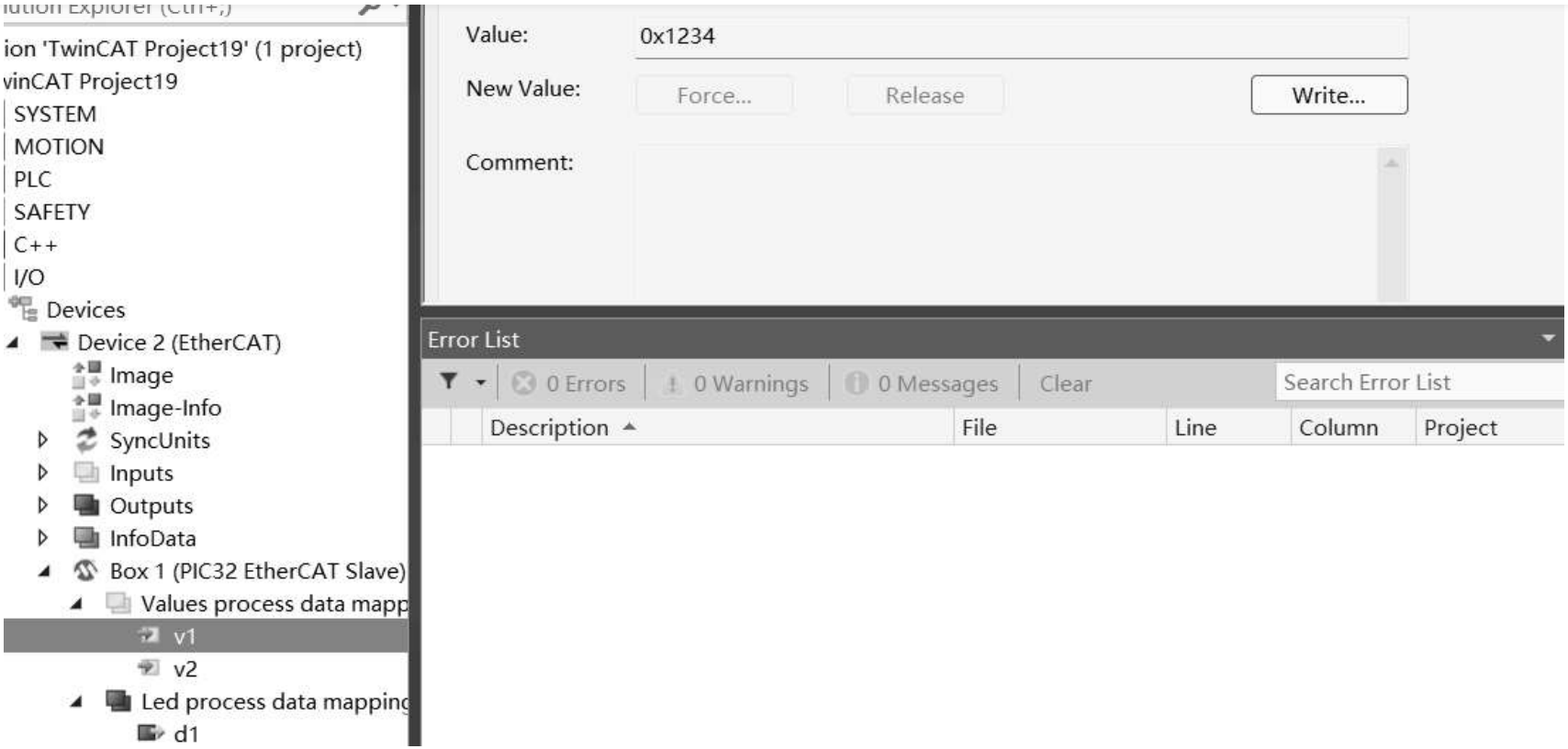
4. 扫描设备，一路确定，等待EEPROM烧写完成

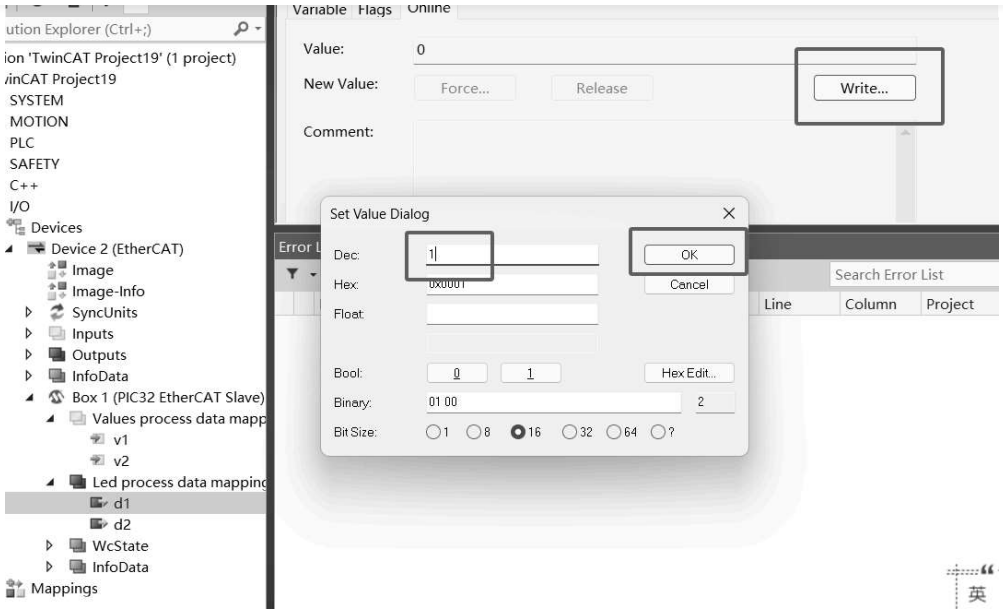




下面这种状态就表示通信成功了

观察是否输出了自己期望的数据以及能否正常控制LED


ion 'TwinCAT Project19' (1 project)
vinCAT Project19
SYSTEM
MOTION
PLC
SAFETY
C++
I/O
Devices
Device 2 (EtherCAT)
Image
Image-Info
SyncUnits
Inputs
Outputs
InfoData
Box 1 (PIC32 EtherCAT Slave)
Values process data mapp
v1
v2
Led process data mapping
d1
Value:
0x1234
New Value:
Force...
Release
Write...
Comment:
Error List
0 Errors
0 Warnings
0 Messages
Clear
Search Error List
Description
File
Line
Column
Project



到此就恭喜各位完成了STM32+LAN9252从站的搭建。

本文章已经生成可运行项目
一键运行

告诉 InsCode 你的需求，生成一个功能完整的网站...
生成项目

显示推荐内容